Toward Web Test Case Generation from Task Descriptions via Iterative Agent Planning

ANONYMOUS AUTHOR(S)

In automated web testing, generating test scripts from natural-language task descriptions is crucial for enhancing the test generation. This activity involves creating the correct sequences of actions to form test scripts for future testing activities. Current state-of-the-art approaches are limited in producing *the sequences of executable actions* due to the focus on task completion, rather than test script generation. Other approaches either demand substantial manual effort for human demonstrations or rely only on re-planning with reflections, which focus on reactive plan repairing. In this paper, we introduce HxAgent, an iterative large language model agent planning approach that uses a proactive correction strategy, iteratively reassessing the state after each step. It determines the next action based on: 1) observations of the current contents and feasible actions, 2) short-term memory of previous web states and actions, and 3) long-term experience with (in)correct action sequences. The agent generates a sequence of actions to perform a given task, which is effectively an automated test case to verify the task. We conducted an empirical evaluation of HxAgent in multiple datasets. On the MiniWoB++ dataset, our approach achieves **97.4%** `Exact-Match` accuracy that is comparable to the best baselines while eliminating the need for human demonstrations required by those methods. On our collected dataset with 350 task instances across seven websites, HxAgent achieves high performance, with **83.8%** of the action sequences exactly matching the ground truth and a `Prefix-Match` of **91.8%**, outperforming the baseline by 11.4%. On OnlineMind2Web, HxAgent achieves 50.7% `Exact-Match`, outperforms the best baseline by 19.3%.

CCS Concepts: • **Computing methodologies** → **Neural networks**; • **Software and its engineering**;

Additional Key Words and Phrases: Web UI Testing, Test Script Generation, Large Language Models

ACM Reference Format:
Anonymous Author(s). 2018. Toward Web Test Case Generation from Task Descriptions via Iterative Agent Planning. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 22 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In Web user interface (UI) testing, part of the process is still manual and error-prone. A tester is provided with the goal of a task in texts that testers would perform on the target application. Testing tools (e.g., Selenium [55], Katalon [30]) are used to launch an application in a controlled test environment, which facilitates interactions with the application by simulating user actions, e.g., clicking buttons, entering text, and navigating between pages. As the tester manually performs the task, the test environment records each step. The recorded sequence of actions (hereafter SoA) corresponds to a test case, complete with its automated scripts produced by the test environment. Manually performing and recording actions for every test case can be time-consuming and prone to human error. Automating this accelerates and scales up test creation while reducing mistakes.

With advances in machine learning and large language models (LLMs), LLM-based solutions have emerged [15, 27, 29, 38, 39, 59], but they **cannot directly support test script generation**.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY
© 2018 ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

LLM planning approaches in **UI agents** interact with Web applications to achieve task goals. However, the UI agents in this category have the following types of issues. First, the models in WebArena [73] and WebLinx [46] *prioritize task completion* over capturing the corresponding sequences of actions (SoAs) that are critical for generating reproducible, executable test scripts. These approaches are primarily aimed at end-user task automation rather than automated test generation, and therefore *do not directly derive executable actions*. Second, other visual planning approaches produce SoAs by reasoning over visual content (e.g., SeeClick [10], UI-TARS [52]), or multi-modal information (e.g., SeeAct [70], CogAgent [25]). However, these methods often model actions using *absolute screen coordinates* extracted from screenshots—a fragile locator strategy for regression testing as minor UI changes can invalidate coordinate references. Consequently, translating an *action description* tied to absolute screenshot coordinates into an *executable action* suitable for regression test scripts remains a challenging problem [70]. SeeAct [70] enables the prompt to the GPT-4V to derive the executable action, while Guardian [53] parses the LLM's response for executable actions. Both suffer low accuracy due to the conversion from task description into executable actions. Third, other UI agents (e.g., MindAct [14], RCI [31], AdaPlanner [60], Synapse [71]) leverage the ability of LLMs to recognize and follow patterns of actions, however, *heavily rely on human demonstrations (examplar actions)*. However, they require a lot of manual effort. Fourth, to avoid human demonstrations, Li *et al.* [35] iteratively progresses a task by identifying and learning from its mistakes through *self-reflection* and *structured thoughts*. However, reflection may enter *trial-and-error loops* with no guidance and *no reasoning on the previous state-action pairs*.

Instead of end-to-end, staged planning as in SOTAs, we propose HxAgent, an **iterative planning** approach to automate the generation of SoA to complete a task in a web application from a given goal description. Inspired by ReAct planning [67], we introduce an iterative LLM-based planning scheme that operates in a continuous loop, *reasoning* to make decisions about the next *action* on the current page based on the following *observations* of: 1) the current page content, 2) feasible actions available, 3) the history of state-action pairs, and 4) experience of both correct and incorrect sequences from the past attempts. The chosen action is executed on the current page by a testing environment, and the new page content informs the next iteration until the task is completed.

In general, instead of letting an LLM autonomously perform the navigation, *our iterative design systematically* **anchors the model's reasoning in the textual and visual evidence available at each step**, *thereby reducing the likelihood of generating irrelevant or fabricated actions*. By *enforcing a grounding mechanism that restricts decisions to screen-level context*, HxAgent provides stronger reliability and robustness, particularly in cases where UI agents are susceptible to hallucinations.

HxAgent advances the state-of-the-art approaches in the following. First, with iterative ReAct planning, it adopts a strategy of **proactive correction**, continuously reassessing the state after each step to adjust its plan as needed. In contrast, *the state-of-the-art self-reflection approaches use a reactive repair strategy, where corrections are attempted only after an entire action trajectory has failed*. Our *proactive correction could help reduce potential hallucinations* from the state-of-the-art LLM planning approaches. Second, instead of suggesting arbitrary actions, HxAgent uses a list of currently feasible actions extracted from the page (e.g., clickable elements, text fields) to ensures the agent only considers *executable and relevant actions*. Third, HxAgent tracks the short-term memory of previous state-action pairs, allowing it to reason over prior attempts, enhancing the agent's understanding of the structure of the task, not just isolated steps. Finally, HxAgent combines the short-term memory of past actions and the long-term experience from both correct and incorrect SoAs, allowing future action decisions to be more informed rather than trial-and-error.

Our empirical results demonstrate that HxAgent achieves substantially higher `Exact-Match` accuracy than the best baselines, outperforming them by **19.3%**, **5.7%**, and **11.4%** on three large web

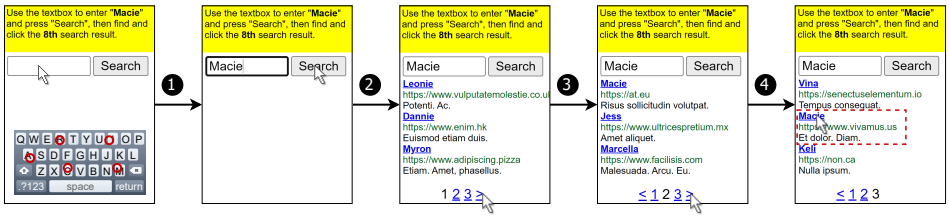


Fig. 1. Step-by-step actions of an example `search-engine` task in the MiniWoB++ dataset [38].

task navigation datasets OnlineMind2Web, MiniWoB++, and the Real-world dataset, respectively. Moreover, HxAgent is capable of automatically generating test scripts, achieving success rates of **77.3%**, **100.0%**, and **90.8%** across these datasets. On particularly challenging tasks, HxAgent attains 81.6% `Exact-Match` accuracy, while the baselines largely struggle with them.

In brief, this paper provides the following contributions:

1. HxAgent: an LLM-based approach to support automated web testing by generating the sequences of actions to perform on a web application to accomplish a given task.

2. An iterative LLM-based agent planning with both short-term and long-term experience.

3. An empirical evaluation to show its effectiveness over the baselines and **a dataset** [28] containing tasks and actual sequences of actions, which can serve as a benchmark for future studies.

2 Motivation Example

An important step in web testing is generating a sequence of actions on the web pages from a natural language description of a task. Fig. 1 displays the task with the description: *"Use the textbox to enter 'Macie' and press 'Search', then find and click on the 8th search result."* The tester is expected to follow this description and perform the described actions on the web pages under test. The tester uses a testing environment (Selenium or Katalon) to execute the application. First, the tester must type "Macie" into the textbox and click the "Search" button. They then need to click the "≥" hyperlink twice before selecting the hyperlink associated with the entry "Macie" as shown in Step 4. The process of the tester's performing a SoA (e.g., clicks, inputs) to interact with the environment (browser), ultimately changes the application's states (e.g., GUI, HTML, or screenshot).

2.1 State-of-the-art Approaches

First, several LLM-based solutions have emerged to automate the manual process [19]. First, the **UI agents for Web task execution** such as UI-TARS [52], SeeClick [9], SeeAct [70], CogAgent [24], and the models used in WebArena [73] and WebLinx [46], focus on completing the task of "clicking on the 8th search result", i.e., the link with the word "Macie" (Fig. 1). They *do not explicitly generate the executable actions that are needed for test script generation for Web applications.* For example, the WebArena benchmark would check only whether a model finally clicks on the "Macie" link.

Second, in contrast, UI-TARS [52], SeeClick [9], and CogAgent [24] generate the sequences of actions (SoAs) where an action is expressed as an operation on an screen area defined by **absolute coordinates**, e.g., a "click" on [0,25,75,90] for the "Macie" link. *This is an unreliable locator approach for regression testing*, since even small UI modifications (e.g., shifting, resizing, or adding elements) can break coordinate-based references. It is not trivial to transform an action description with absolute coordinates into executable actions that are suitable for regression test script generation.

Third, another prominent line of approaches describe **the actions or SoA in natural language**. In our example, at step 4, SeeAct [70] could describe the action as *"click on the word "Macie""*. However, there are two appearances of that word on the screen after step 4, which might make the model confused. In fact, it was reported that SeeAct suffers much lower accuracy in SoA generation when outputting executable actions [70], which are crucial for Web test script generation.

Fourth, several **prompt-based approaches with demonstrations** such as MindAct [14], RCI [31], AdaPlanner [60], and Synapse [71] leverage *few-shot or many-shot prompting* by providing exemplars to the LLMs. In our motivating example (Fig. 1), a demonstration will contain a sequence of actions including typing a word into a search box, clicking on the "Search" button, clicking on a link for pagination, and clicking on a resulting link. However, these methods face two key limitations: (1) they demand substantial manual effort to create demonstrations on web UIs, and (2) they risk potential overfitting to the demonstrated patterns.

Fifth, the prompt-based approaches without demonstrations leverage the **self-reflection** capability of LLMs to identify and revise the first critical mistake after a failed attempt, and retry the task. However, *they operate in a reactive repairing strategy, where corrections are attempted only after an entire action trajectory has failed.* Moreover, without maintaining a record of prior state-action pairs, for instance, at step 3 in Fig. 1, the model might mistakenly click the hyperlink labeled "1," returning to the previous screen. A heuristic based on textual similarity (e.g., matching the word "Macie") fails, as multiple links share that label in steps 3 and 4. Relying on DOM indices is also unreliable, as not all search results are visible on a single page due to pagination.

2.2 Key Ideas

We introduce HxAgent with the following key ideas:

*2.2.1 Key Idea 1 [***Iterative LLM-based Agent Planning****].* Unlike the autonomous UI agents, to *explicitly model the generated sequences of executable actions*, we develop an *iterative LLM-based agent planning* that controls the next action in a continuous loop. **Our iterative process keeps the model grounded in the screen's text and visuals***, preventing irrelevant or hallucinated actions.* HxAgent makes decisions on the actions to be performed on the current page based on the observations of 1) the current page, 2) feasible actions on the page, 3) the sequence actions memory consisting of the sequence of previous state-action pairs, and 4) the experience of (in)correct sequences in the history. This chosen action is executed on the current page by a testing environment, and the resulting new page content is used for the next iteration until the given task is considered as accomplished. Unlike the UI agent approaches, an action is explicitly represented as a tuple of an operation on an Web element with optional input values. Compared with the self-reflection direction, HxAgent follows **proactive correction** of a plan where it can correct course during execution by constantly reassessing the situation after each step. In contrast, self-reflection performs reactive repair only after a task fails. This delays error correction, *only attempting to repair after an entire trajectory fails*, and increases the chance of wasting effort on a faulty plan before any feedback is used.

*2.2.2 Key Idea 2 [***Sequence actions memory** *as short-term memory].* A crucial aspect of navigating complex web tasks is maintaining a persistent awareness of previous steps and the evolving states of webpages. Drawing inspiration from this necessity, we introduce a short-term memory mechanism that involves feeding the LLMs with information about the previous steps taken and the corresponding states of the website after each action (in terms of json files and xpath locators). HxAgent maintains a structured memory of prior interactions, allowing it to avoid previously failed decisions and inform future reasoning with explicit historical context.

*2.2.3 Key Idea 3 [***Combining action memory with experience reinforcement****].* Building upon LLMs' reflection [57] as in [35], we propose a two-phased strategy consisting of a training phase and an evaluation phase. During the training phase, LLMs rely on their reflection capabilities to gather the entire sequences of prior state-action pairs the corresponding correct/incorrect status. The knowledge from this phase is aggregated to enhance performance during the evaluation phase. In practice, the judgment of sequences can be part of the process where testers evaluate them.

Short-term memory complements experience to enhance the ability of the LLM to learn from recent interactions and make informed decisions for future actions. Short-term memory enables the model to retain information about the most recent actions and their outcomes, providing essential context and continuity. For instance, when navigating a series of webpages to complete a task, short-term memory allows the LLM to remember recent clicks, form inputs, and navigation choices. Meanwhile, experience draws on a broader set of patterns and knowledge from past interactions, helping the model recognize similarities with previous tasks and apply effective strategies.

2.2.4 Key Idea 4 ***[Vision enhancement]****.* On real-world websites, the states of the websites represented as HTML can be extensive. This behavior can lead to exceeding the context length of LLMs. Moreover, selecting elements for interaction can be challenging, especially when complex webpage layouts make it difficult to fully extract the element's context. For example, in a date picker, the algorithm might detect "Day 1" as interactable but fail to distinguish between different months. This ambiguity results in confusing options. To address them, we use the multi-modal capabilities of LLMs by providing them with the screenshot and actionable elements of the current webpage. These inputs along with others support LLMs in choosing the next action.

3 Architecture Overview

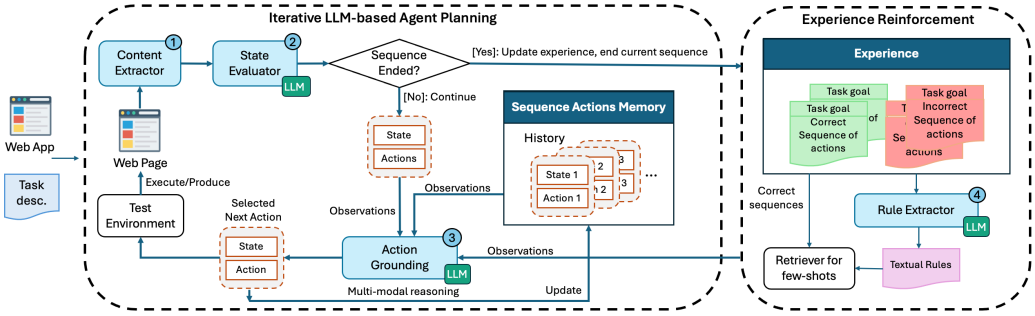


Fig. 2. HxAgent: Architecture Overview.

Sequence Generation. Fig. 2 shows HxAgent's architecture overview with LLM indicating LLM-based agents. It takes as input the application under test and the task description, and produces the SoA on the corresponding webpages to finish the task. HxAgent operates with two key components: 1) Iterative LLM-based Agent Planning to produce action sequences, and 2) Experience Reinforcement to support the decision-making.

Given a target web application and task description, HxAgent initiates its iterative process from the specified starting page URL. The process begins with the Content Extractor 1, which operates on the current webpage to capture the HTML content and enumerate feasible actions in JSON format. Next, the State Evaluator 2 employs LLMs to analyze the current state of the action sequence. Within a single prompt, the agent (i) summarizes previous interactions for the task, (ii) describes the current state, (iii) determines whether to stop execution early due to significant divergence from the expected state, (iv) decides whether the task is complete, and (v) evaluates whether the intended goal has been successfully achieved.

Inspired by ReAct planning framework [67], which has been successfully applied in robotics and other domains by instructing the LLM to pause execution after each cycle of reasoning and actions to observe the environment, HxAgent performs as follows. At each iteration i, the Action Grounding agent 3 performs reasoning $\mathcal{R}$ to decide the next action based on three key external observations O. The first observation O_1 is the current webpage and its corresponding actions

produced by the content extractor. The second observation O_2 is the history of actions, which is the sequence of state-action pairs where states are the web contents. The third observation O_3 is the experience from previous action sequences that are judged as correct or incorrect with respect to the task description. Mathematically, the next action on the corresponding webpage a_{i+1} is determined via $\mathcal{R}(O_1, O_2, O_3)$. The chosen next action in the form of a state-action pair is used as input for a testing environment (e.g., Selenium) that executes the application at the selected webpage to produce the next page for the following iteration. This iterative planning continues until State Evaluator 2 decides that the task is accomplished or there is no way to accomplish the required task.

Short-term memory maintains immediate context and the impact of recent actions, while experience offers learned strategies. For sequence action memory, the history of the SoA is recorded as the process progresses. HxAgent maintains this history as a list of state-action pairs, with states represented as HTML text and actions as JSON text. This functions as a short-term memory, enabling LLM to maintain persistent awareness of recent actions and the evolving states of the current webpage. After choosing the action for the next iteration, it updates this memory.

For experience reinforcement, researchers [57] have demonstrated that reflection-based experience can help an LLM improve its performance through reinforcement learning with examples. Specifically, we enhance the central LLM's reasoning performance by providing the LLM with few-shot examples of correct action sequences and textual rules that summarize past correct or incorrect sequences. These rules are generated by Rule Extractor 4 from previous action sequences. For example, a rule might be: "Ensure to verify the text of the element before clicking, to confirm if it is the desired link and avoid unnecessary clicks on unrelated elements."

Test case generation. HxAgent produces test scripts by converting an action sequence that successfully completes a task into a test case, where the task description serves as the test case name. Each step in the test case corresponds to an action executed on a given state. An action is defined by its type (e.g., click, text entry), value (e.g., input text), target web element, and the state of a webpage. The test cases can be automatically replayed in the target environment during regression testing.

4 Iterative Agent Planning

Algorithm 1 displays the pseudo-code of our iterative LLM-based Agent Planning algorithm. It takes as input a natural-language task description *task* and a webpage *url* for the process. First, the Experience is initialized as empty (line 1). Similarly, the Sequence Actions Memory (*SAM*) is also initialized for the given task with an empty state-action history (line 3). It uses a test environment to parse the current webpage to extract the DOM (line 4), which is then passed to the Content Extractor to obtain the initial state and the list of feasible actions (line 5). If the web content contains several irrelevant, we utilize the LLM to summarize the web screenshot content into a textual form along with the feasible actions. While the task is not accomplished or the number of steps does not reach a limit yet, we perform the iterative planning process (lines 6–25). At line 7, the LLM is called for reasoning $\mathcal{R}$ to derive the next action based on the observations O including 1) the current state s_i and feasible actions A_i (O_1), 2) Sequence Actions Memory *SAM* (O_2), and 3) Experience (O_3). We will explain the prompts later. If this action is duplicated, meaning there is an identical action in the current feasible actions A_i, the LLM is called again to select the correct candidate from the duplicated pool by utilizing the screenshot of the current state img_i (lines 8–10). The vision capability of the LLM is leveraged to fully capture the context of actions where the Content Extractor produces duplicated elements. If this action signals a *Done* flag by the LLM, we finish the loop and update the Experience with our generated SoA to enhance the next iteration (lines 11–16).

This update on Experience is performed only in the training phase after the SoA is judged by testers (in our experiment, we use the ground truth for the judgement). If this action is an input

Algorithm 1 Iterative LLM-based Agent Planning

```
1: Input: a task to complete and an url of intended webpage
2: Exp ← Experience()
3: procedure IterativeAgentPlanning(task, url)
4:     SAM ← SequenceActionMemory(task)
5:     page ← TestEnvironment.load(url)
6:     s_1, A_1 ← ContentExtractor(page)
7:     while step limit not reached do
8:         a_i ← LLM(s_i, A_i, SAM, Exp)
9:         if a_i is duplicated then
10:            a_i ← LLM(img_i, A_duplicated, SAM, Exp)
11:        end if
12:        if a_i finalizes the task then
13:            if is in training phase then
14:                Exp.update(SAM)
15:            end if
16:            break
17:        end if
18:        if a_i is input operation then
19:            content ← LLM(SAM, Exp)
20:            a_i.extend(content)
21:        end if
22:        SAM.append({s_i, a_i})
23:        page ← TestEnvironment.execute(a_i)
24:        i ← i + 1
25:        s_i, A_i ← ContentExtractor(page)
26:    end while
27: end procedure
```

action, we then ask the LLM to generate the content for the target field (lines 17–20). Finally, we update Sequence Actions Memory with a state-action pair (line 21). The Test Environment will execute the action on the web application to get the updated webpage (line 22) and get the updated state and feasible actions for the next iteration (lines 23–24).

Let us take the iteration #4 of Fig. 1 as an example. The HTML content of the webpage and its action transition arrow represent a state-action pair. At the end of iteration 3, *SAM* currently stores the state-action pairs of the history including the ones at 1) the iteration i=1 (No search results displayed, input text: "Macie"), 2) the iteration i=2 (No search results displayed, input text: "Macie", clicking "Search"), and 3) the iteration i=3 (three links displayed: "Leonie", "Dannie", "Myron", pagination links, and clicking the "≤" hyperlink). Then, an action a_3 is performed to transit the 3rd page to the 4th. Next, we extract the current state s_4 and feasible actions A_4 (including the input search field, clicking Search button, clicking the "Macie", "Jess", and "Marcella" hyperlinks, clicking the "≤" hyperlink, etc.). At the iteration $i = 4$, the LLM reasons and decides the next action a_4 to be "clicking the "≥" hyperlink" based on the observations of current webpage state s_4 and its corresponding actions A_4, the sequence actions memory *SAM* up to iteration i_3, and the experience E. Then the procedure at the end of iteration $i = 4$ continues and moves on to $i = 5$ until the task is finished. Due to maintaining the contents/states of the steps, it does not stop and click at the "Macie" hyperlink at the 4th search result, and continues to "Macie" link in the next iteration.

4.1 Task Description & Content Extractor

```
The search results are displayed in a div with the id 'page-content'.
The first three search results are: 'Leonie', 'Dannie', and 'Myron' with additional links and descriptions.
There is also a pagination section with links to navigate through the pages.
```

Fig. 3. State in summarized natural language description format extracted from 3rd webpage of Figure 1.

```
1  ITERATIVE_ACTION_REASONING_PROMPT = '''You are a web assistant... You will complete the task by taking a
       series of steps. Each step is a description of the action you take and the specific item, entity, or
       element on the website that the action is applied.
2  {experience}
3  # Here is the actual task.
4  {sequence_actions_memory}
5  After completing the above steps, you reach a state: {state} where the following feasible steps exist:
6  {feasible_actions}
7  POSSIBLE NEXT ACTION #1: {action_1} ...
8  Your job is to choose the most possible next steps to help you complete the task....
9  # The JSON response must strictly follow these rules:
10     "chosen_action": ... (the index of the potential action that you choose)
11     "action_description": ... (a string describing the action you choose)
12     "reason": (describing why you choose the action)...
```

Fig. 4. Iterative Action Reasoning Prompt (Main Prompt).

The input task is described in natural language. This description is recorded in the Sequence Actions Memory (line 3, Algorithm 1). HxAgent utilizes Content Extractor 1 (lines 4–5, , Algorithm 1) to extract information from the initial webpage and update the information after the agent performs actions (lines 22–24, Algorithm 1). The webpage is passed to the LLM in the DOM format. This module has two functions: (1) feasible action extraction and (2) state extraction.

First, the LLM receives the DOM of the page and generates possible actions described as JSON objects. We extract only the visible and interactable HTML elements attached to specific user events using the DevTools API. We also extract relevant surrounding contexts. For example, the context for a button and a container could be its inner content, while an input field's context could be its label. If necessary, we traverse up the DOM tree to gather more information from parent nodes and bind accordingly. After that, each interactable element is mapped with its operation, resulting in a list of possible actions on the current webpage in a JSON file.

Second, HxAgent extracts and represents the state of a webpage in one of two formats, based on the DOM size. For smaller DOMs, it generates a simplified HTML containing interactable elements and a mapping to the original HTML file. For large DOMs that exceed the LLM's context, it produces a text summary generated by the LLM from the webpage screenshot using vision capability (Fig. 3).

4.2 Sequence Actions Memory (SAM)

This module acts as a part of Observation (O_2) to inform the LLM on the previous steps and the evolving states of the webpages. It is first initialized with the task description (line 3, Algorithm 1) to keep track of the task we are operating during the iterative agent planning. At each iteration i, the pair of the current page in the form of state s_i and an action a_i to be performed chosen from a list of feasible actions A_i are updated into SAM (line 21, Algorithm 1) to act as a short-term memory of previous state-action pair for the next cycle.

4.3 Reasoning with LLM Agent

We use Action Grounding 3 as a reasoning component, $\mathcal{R}$. It processes a stream of grounding observations to determine the next action from the action space at each iteration.

The observations O consist of three key inputs, converted into text for prompt construction for the LLM (Fig. 4). The first observation, O_1, is the current webpage and its corresponding actions

```
SEQUENCE_ACTIONS_MEMORY_PROMPT = '''
You are visiting the website title: {title}
You are asked to complete the following task:
      {task}
You have completed the following steps:
> STATE #1: {state_1}
> STEP #1: {action_1}
> STATE #2: {state_2}
> STEP #2: {action_2} ...
```

Fig. 5. Sequence Actions Memory Section of Main Prompt.

```
EXPERIENCE_PROMPT = '''
# Here are the history of your trials
SUCCESS TRIAL #1: Task: {task_1}
STEP #1: {task_1_action_1}
STEP #2: {task_1_action_2}
...
SUCCESS TRIAL #8: Task: {task_8}
STEP #1: {task_8_action_1}
STEP #2: {task_8_action_2}
...
# Rules extracted from past attempts, use to
      evaluate your policy:
RULE #1: {rule_1}
RULE #2: {rule_2}...
```

Fig. 6. Experience Section of Main Prompt.

produced by the content extractor. We found that focusing on the target elements of the page in each feasible action along with the summarized state sufficiently represents the current page, so we construct a section in the main prompt containing feasible JSON object actions and natural-language state for O_1. The second observation, O_2, is the SAM embedded in another section of the main prompt (Fig. 5), containing a sequence of state-action pairs where states are represented in HTML or summarized text format. The third observation, O_3, is the experience from previous action sequences (Fig. 6), forming a section in the main prompt.

Mathematically, the action space is the list of feasible actions from O_1. With the observations and available action space, we perform a reasoning process. Specifically, for a task τ, we have these observations: the current state with action space O_1, the sequence actions memory O_2, and the experience O_3. At iteration i, we obtain the current state with k feasible actions $O_1^i = \{s_i, a_0, ..., a_k\}$ and short-term memory $O_2^i = \{\tau, (s_0, a_0), ..., (s_i, a_i)\}$. We then use the prompted LLM to determine the next action a_{i+1}:

$$a_{i+1} \sim \mathcal{R}_{LLM}(\cdot \mid O_1^i, O_2^i, O_3) \tag{1}$$

where $\mathcal{R}_{LLM}$ denotes the LLM reasoning model.

5 Experience Reinforcement

Training phase. Initially, the experience is set to empty. During the training phase, the experience is updated once the iterative agent planning signals a Done flag. For update, the (in)correctly generated sequences of actions are saved as experience. For an incorrect sequence, we Rule Extractor 4 to extract a textual rule from the (in)correct sequence, ensuring it differs from the current rules, and update it to the current set of rules. These rules act as textual heuristics, along with the correct SoA serving as few-shot examples embedded in the experience section of the main prompt (Fig. 6). This part of the main prompt is dynamically updated in the training phase. We also track the average of the correct SoA generated over a number of recent episodes in training (called *moving average*). Consider Experience Reinforcement with task `click-tab-2`. The first episode initializes the task: "*Switch between the tabs to find and click the link 'et'.*" HxAgent uses Iterative LLM-based Agent Planning to generate and execute the action "*click the link 'et'*," completing the task. The correct sequence is then stored in the Experience (Algorithm 1, lines 12–15). In the second episode, a similar task is initialized. The Experience guides the agent, but this time the generated sequence is incorrect. It is saved as an incorrect sequence, and Rule Extractor creates the rules. In the third episode, with updated Experience (correct sequences+rules), it *switches tabs* before clicking the link, creating a correct sequence. This is added to Experience.

Evaluation phase. We select the optimal experience from training to be used as input for the evaluation phase. An optimal experience is the combination of the correct SoA and rules that

Table 1. Real-world dataset statistics.

Application	#Actions/Screens	#Screens	SoAs
YouTube	291 ± 144	2 ± 1	3 ± 1
LinkedIn	146 ± 36	3 ± 1	4 ± 1
Facebook	260 ± 110	3 ± 1	3 ± 1
Google	91 ± 189	3 ± 1	4 ± 1
Amazon	330 ± 82	2 ± 1	3 ± 1
StackOverflow	296 ± 151	3 ± 1	5 ± 5
Expedia	135 ± 22	4 ± 1	9 ± 4
Total/Avg.	221	3	4

Table 2. Baseline approaches

Approach	Model	demos	Feedback
WebNT5 [21]	T5+Fine-tuning	✓	-
HTML-T5-XL [20]	T5+Fine-tuning	✓	-
WebGUM [17]	ViT+T5+Fine-tuning	✓	-
WGE [38]	RL	✓	-
CC-Net [27]	RL + SL	✓	-
RCI [31]	LLM Prompting	✓	✓
AdaPlanner [60]	LLM Prompting	✓	✓
Synapse [71]	LLM Prompting	✓	-
Li et al. [35]	LLM Prompting	-	✓
SeeAct [70]	LLM Prompting	-	✓

exhibits an upward trend and stable pattern from the training, decided by the moving average. The iterative planning is the same, except for the fixed optimal experience O_3^{opt}. Hence, with experience, the reasoning for the next action a_{i+1} at the iteration i become:

$$a_{i+1} \sim \mathcal{R}_{LLM}(\cdot \mid O_1^i, O_2^i, O_3^{opt}) \tag{2}$$

6 Experimental Methodology

For evaluation, we seek to answer the following questions:

RQ1. [Effectiveness on Action Sequence Generation]. To what extent does HxAgent generate sequences of actions (SoAs) for completing tasks?

RQ2. [Test Script Generation Effectiveness]. How well is HxAgent in generating test cases?

RQ3. [Experience Analysis]. How well does Experience enhance HxAgent's performance?

RQ4. [Ablation Study]. What is the impact of each HxAgent's component on its performance?

RQ5. [Cost]. What is the cost of using the LLMs?

Datasets. We use three datasets for evaluation. The first, the Real-world dataset (Table 1), was collected from a set of well-known websites including YouTube, LinkedIn, Facebook, Google, Amazon, StackOverflow, and Expedia. For each website *url*, we designed a set of task templates (e.g., *"Subscribe to the {} channel"*, *"Login to YouTube with username {} and password {}"*). Each template was then instantiated into specific tasks (e.g., *"Subscribe to the 'TEDx Talks' channel"*). The final dataset consists of 50 task instances per website across 7 websites, yielding a total of 350 task instances.

The second dataset is from the **MiniWoB++** (Mini World of Bits++) benchmark [38]. Each environment or application is defined for a task, having a description (e.g., *"Choose an item from a list"*), a set of utterances (e.g., *"Bobine"*, *"Betty"*) that were randomly chosen for each episode to form a complete natural language task instance (e.g., *"Select Betty from the list..."*). We sampled 44 tasks from the dataset, each with 25 instances, yielding a total of 1,100 task instances.

The third dataset is **OnlineMind2Web** [65], a recent and actively maintained benchmark derived from the original Mind2Web dataset [14]. It contains 300 tasks collected from 136 real-world websites across domains such as clothing, food, housing, and transportation. The dataset classifies tasks into three difficulty levels: *easy* for those requiring up to 5 steps to complete (83 tasks), *medium* for those requiring 6–10 steps (143 tasks), and *hard* for those involving 11 steps or more (74 tasks). In our experiments, 15 tasks were excluded because four websites were inaccessible, resulting in a final set of 285 tasks from 132 websites used for evaluation.

Baselines. We used 10 baselines (Table 2). The first eight baselines including WebNT5, HTML-T5-XL, WebGUM, WGE, CC-Net, RCI, AdaPlanner, and Synapse were evaluated exclusively on

Table 3. Performance of Sequence Action Generation on Real-world dataset (RQ1).

Website	Exact-Match (%)			Prefix-Match (%)		
	HxAgent	Li *et al.*	SeeAct	HxAgent	Li *et al.*	SeeAct
YouTube	100.0	20.0	100.0	100.0	20.0	100.0
Linkedin	92.0	46.0	100.0	98.4	71.0	100.0
Facebook	93.3	33.3	66.7	96.7	39.0	79.2
Google	96.0	0.0	100.0	98.6	0.0	100.0
Amazon	93.4	54.0	100.0	94.4	56.0	100.0
Stackoverflow	80.0	36.0	40.0	90.0	45.4	63.3
Expedia	32.0	0.0	0.0	64.4	9.4	44.1
Total/Avg.	**83.8 (± 32.6)**	27.0 (± 35.8)	**72.4 (± 42.2)**	**91.8 (± 19.8)**	34.4 (± 28.3)	**83.8 (± 43.9)**

MiniWoB++, as these methods depend on human demonstrations, which are unavailable in the other two datasets. For SeeAct, we use ($SeeAct_{choice}$), which grounds actions via textual choices. Refer to the Related Work section for the rationale behind our selection of benchmarks and baselines.

Procedure. We set up Selenium test environment via ChromeDriver. The task in the text string is used as input to HxAgent and baselines. We use *GPT–4o* in our experiments, as it serves as the primary LLM in most baselines and is widely regarded as a state-of-the-art model.

For MiniWoB++, we first run the training phase for 20 task instances, to obtain the maximum of 8 few-shot examples and a set of textual rules to be used as experience. As in Li *et al.*, we evaluated each task on 50 instances. For real-world dataset, we evaluated on 50 instances for each task.

The results reported in the original Li *et al.* paper are based on *PaLM*, which has been discontinued. Thus, we ran the Le *et al.*'s experiment using the codebase with OpenAI's *GPT-4o*.

Metrics. We define an action as correct if both the selected element (its content) and the predicted operation are correct. For sequence action generation, we use:

Exact Match Rate (`Exact-Match`*)*: We consider the generated SoA for a task as correct only if all actions are correct. `Exact-Match` is defined as the ratio of correct results to the total number of results.

Prefix Match Rate (`Prefix-Match`*)* `Prefix-Accuracy` is the accuracy of the prefix of a generated SoA for a task instance, which is the ratio of continuous correct actions from the start to the first error, divided by the total number of actions. `Prefix-Match` is the average in all instances.

7 Empirical Results

7.1 Effectiveness in Sequence Action Generation (RQ1)

7.1.1 **Performance on Real-world dataset.** As seen in Table 3, HxAgent achieves an average of 83.8% `Exact-Match` and 91.8% `Prefix-Match` across all task instances. HxAgent outperforms Li *et al.* by 56.8%, with the gap being most significant in tasks that require navigating more than two screens and completing at least two actions. In this real-world dataset, websites are more dynamic than those in MiniWoB++, particularly when dealing with dropdown options (e.g., search tasks in `Google` or filling in locations to book flights on `Expedia`, Table 3). These dynamic features pose a challenge for the staged planning approach of Li *et al.*, which plans all actions on a screen at once. The reflection alone in Li *et al.* learns from the entire passing/failing sequences, thus, less flexible than its combination with ReAct and short-term memory as in HxAgent. Large state changes by an action often lead to the failure of subsequent actions in the plan. Moreover, Li *et al.* use the entire HTML for state representation, resulting in exceeding LLM context or selecting invalid actions.

In contrast, HxAgent performs well on the tasks in this dataset. These tasks involve state changes after an action, making the iterative planning in ReAct effective in adapting to changes. These tasks involve navigating up to four screens, performing up to five actions, processing large HTML files.

However, some tasks still pose greater challenges due to highly dynamic layouts and varying action sequences across runs. For instance, ordering a gift card may require navigating to the product detail page and filling out a "Gift Card Details" form, whereas ordering a smartphone might only require clicking "Add to Cart" after searching. These variations in layout/actions make planning with prior experience more difficult, leading to occasional inaccuracies. For `Expedia` tasks, it achieves the lowest `Exact-Match` of 32% and `Prefix-Match` score of 64.4%. It struggles to handle real-world date pickers to select departure and return dates. HxAgent shows a performance with 80% `Exact-Match` and 90% `Prefix-Match` in `Stackoverflow` tasks. There is one task that requires filling a long form with constraints, resulting in an incorrect actions.

We also evaluated the best-performing variant of SeeAct [70], $\texttt{SeeAct}_{choice}$, on the Real-world dataset. This model, which grounds actions using textual choices, was used to assess how a strong web-task model could be adapted to a new dataset. SeeAct achieved an `Exact-Match` rate of 72.4% and a `Prefix-Match` rate of 83.8%. It can handle most tasks on platforms such as `YouTube`, `Linked Google`, and `Amazon`. However, its performance drops to 66.7% on `Facebook`, and it fails to complete tasks on `Expedia` due to difficulties handling real-world date pickers for selecting departure and return dates .

7.1.2 **Performance on OnlineMind2Web.** Table 4 shows the results obtained from running three approaches on the Online-Mind2Web dataset. HxAgent achieves an average of 50.7% average `Exact-Match` and 62.4% `Prefix-Match` across 285 tasks spanning 132 popular websites from various domains. Importantly, it consistently outperforms the baselines across tasks in all difficulty levels. However, its performance varies by difficulty, with 81.3% on easy tasks, 40.7% on medium tasks, and 30.0% on hard tasks. In this dataset, the websites are more complex than those in the Real-world dataset, particularly due to dropdown menus, calendar pickers, and similar components. Some UI elements are clickable while others are disabled, and task completion often depends on executing the correct sequence of clicks, further complicated by errors and formatting artifacts in the UIs. These dynamic features present significant challenges for the staged planning approaches of Li *et al.* and SeeAct. For Li *et al.*, using the DOM to represent the state of a webpage leads to token limitations in the LLM. In contrast, SeeAct suffers from incomplete descriptions of UI elements, which often result in suboptimal action choices.

For easy tasks, HxAgent has relative improvements of 2.03X over Li *et al.* and 27.4% over SeeAct, with the substantial gains occurring in tasks that involve complex interface elements such as duplicate web elements, calendar pickers, and dropdown menus. For example, in the task *"Find Florida internship programs in the Mayo Clinic College of Medicine and Science,"* Li *et al.* was unable to reach the target page, whereas SeeAct failed as it attempted to resolve duplicate "Search" buttons.

For medium tasks, HxAgent achieves relative improvements of 2.75X over Li *et al.* and 33.9% over SeeAct. These tasks require managing state changes triggered by user actions, where HxAgent's iterative planning is particularly effective for adapting to such dynamic interactions. For example, when a user enters a zip code into a search box, the system must wait for the resulting state change and the appearance of a dropdown list before selecting the appropriate item. In the task *"Browse pediatricians near zip code 90028 who specialize in Internal Medicine and have a rating of at least 4 stars,"* after the user inputs "90028" into the Location field, the system must wait for the state update and then correctly select *"Los Angeles, CA 90028"* from the dropdown.

For hard task, HxAgent achieves improvements of 5.3X over Li *et al.* and 3X over SeeAct. These tasks involve complex UI elements such as dropdowns and require multiple steps in the execution process. They also combine characteristics of both easy and medium tasks, as they mix relatively straightforward lookup operations with more intricate interactions that demand careful sequencing of actions. For instance, in the task *"Browse dermatologists within 10 miles of zip code 10019 and filter by only those who accept Blue Medicare Advantage,"* the agent must first enter the zip code "10019"

Table 4. Performance of Sequence Action Generation on OnlineMind2Web dataset (RQ1).

Level	No.	Exact-Match (%)			Prefix-Match (%)		
		HxAgent	Li *et al.*	SeeAct	HxAgent	Li *et al.*	SeeAct
Easy	80	81.3	40.0	63.8	90.8	54.0	75.8
Medium	135	40.7	14.8	30.4	55.0	26.2	42.4
Hard	70	30.0	5.7	10.0	41.2	13.8	20.2
Total/Avg.	285	**50.7**	20.2	34.7	**62.4**	31.3	46.1

Table 5. Action Sequence Generation on MiniWoB++ (RQ1). Exact-match (%), Correct runs (975 total).

Approach	WebNT5	HTML-T5	WebGUM	WGE	CC-Net	RCI	A.Plan.	Synapse	Li (4o)	SeeAct	HxAgent
Correct runs	532	861	877	627	915	926	853	958	949	682	950
`Exact-Match`	59.1%	88.3%	89.9%	64.3%	93.8%	89.5%	88.0%	97.3%	91.7%	69.9%	97.4%

Table 6. Performance on 125 Challenging Task Instances (MiniWoB++).

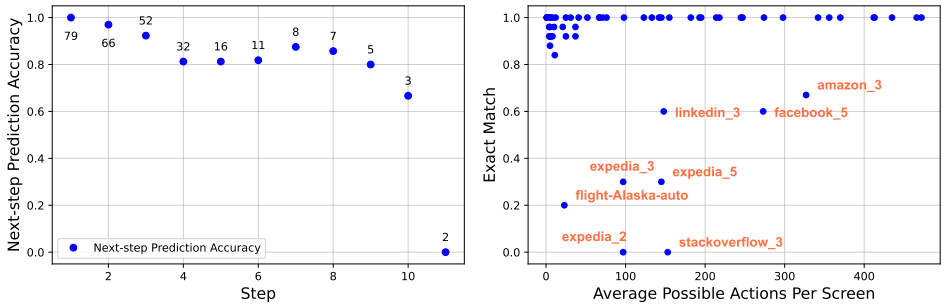
Tasks	#Screens	Actions Seq	Exact-Match (%)		
			HxAgent	Li *et al.*	SeeAct
choose-date	7 ± 5	7 ± 5	92.0	24.0	0.0
book-flight	4 ± 1	7 ± 1	100.0	24.0	0.0
flight.AA	3 ± 0	7 ± 0	100.0	0.0	40.0
flight.Alaska	3 ± 0	9 ± 0	96.0	0.0	0.0
flight.Alaska-auto	4 ± 0	9 ± 2	20.0	0.0	0.0
Total/Avg.	5	7	81.6 (±34.6)	9.6 (±13.1)	8.0 (±18)

into the Location field, wait for the interface to update, and then correctly select *"New York, NY 10019"* from the dropdown before applying the remaining filters. Satisfying the task requirements thus depends on completing the full sequence of steps. Such tasks often unfold as relatively long action chains, typically requiring three to four sub-actions per screen before progressing further.

Although, HxAgent demonstrates substantial improvements over Li *et al.* and SeeAct, HxAgent still struggles with complex components such as datetime pickers, enter-to-submit search boxes, and highly intricate websites that may cause LLM context overflows.

7.1.3 **Performance on MiniWoB++.** As seen in Table 5, which reports 975 task instances from 39 tasks, HxAgent achieves a 97.4% average `Exact-Match` across the shared set of tasks among the baselines. This set of tasks varies in screens and actions, with screens ranging from 1 to 6, and a maximum of 6 actions needed to complete the task. Tasks with fewer screens and actions tend to have a higher average `Exact-Match`, with 97% to 100% reported for tasks with fewer than 3 screens and actions. This percentage drops marginally to 93% for tasks containing more than 3 screens and actions, with a maximum of 4 screens and 9 actions. With `Prefix-Match` of 99%, in many cases where HxAgent does not fully succeed in a task, the generated sequences contain correct prefixes of action sequences, reducing correction effort.

Although HxAgent achieves comparable `Exact-Match` with the best baselines in each category, note that it *eliminates the need for human demonstrations*. Specifically, it matches the performance of WebGUM, despite WebGUM utilizing 401K demonstrations with web screenshots to jointly fine-tune the vision encoder ViT and T5. HxAgent performed better than CC-Net (BC + RL) in 10 task instances, with CC-Net employing 2.4M human's demonstrations. Synapse, with perfect results, relies heavily on 154 quality hand-crafted exemplars, some of which further contain task-specific filter prompts to convert raw HTML states into clean observations. Their solution may result in overfitting MiniWoB++, as they have a high number of few-shot exemplars and the reducing of the difficulty of tasks requiring exploration in multiple states with customized and obvious observation.



(a) Next-step Prediction. (b) Accuracy by Actions/Screens.

Fig. 7. Performance by Task Complexity (RQ1)

Among the approaches that do not rely on human demonstrations, HxAgent performs slightly better than Li *et al.* (97% vs. 94%) and substantially outperforms SeeAct by 20% in `Exact-Match` accuracy. We further stratified results on 125 challenging task instances drawn from five MiniWoB++ tasks (Table 6). These tasks are considered challenging because they require navigation across an average of five screens and the execution of approximately seven actions to complete. As seen in Table 6, HxAgent shows the effectiveness (**81.6% `Exact-Match`**) in more challenging tasks over Li *et al.* and SeeAct, which have only 10% and 8% `Exact-Match` in these complex actions.

As reported in Table 5, SeeAct achieves an `Exact-Match` rate of 69.9%, which underperforms Li *et al.* On the 125 most challenging tasks, its `Exact-Match` rate drops to 8%, with successful completion limited to the `flight.AA` tasks (Table 6). Our further analysis reveals that web components, such as datetime pickers and dropdowns, remain significant obstacles for SeeAct. This is also true for other non-descriptive elements like buttons with only icons, which require action-oriented descriptions to be processed effectively. In brief, *HxAgent achieves higher accuracies on the challenging tasks, while the baselines struggle with such intricate actions.*

7.1.4 **Accuracy by Task Complexity.** We evaluate the performance of HxAgent with respect to task complexity, measured by the length of action sequences and the number of actions on webpages.

Next-step prediction accuracy by action sequence length. In Fig. 7a, HxAgent achieves 100% accuracy on the first step for all tasks in the MiniWoB++ and Real-world datasets. However, as a task requires more action steps, accuracy slightly decreases. (For step 7, a slight increase is observed due to the smaller number of tasks reaching this stage, resulting in a relatively higher value). For two tasks reaching Step 11, it did not predict the correct action, despite the successes of the previous 10 steps. A similar trend is observed in the OnlineMind2Web dataset, where `Exact-Match` drops from 81.3% on easy tasks to 40.7% on medium tasks and 30.0% on hard tasks.

Exact-Match sequence accuracy by number of actions per screen. The action space may contain a large number of selectable actions per screen. As shown in Fig. 7b, HxAgent maintains high `Exact-Match` accuracy even when the action space is large, with up to 500 possible actions per screen.

There are eight outliers (shown in orange): two tasks with sequence lengths of up to 11 steps, where `Exact-Match` falls below 70%. For tasks involving dynamic content, such as confirmation popups in `linkedin-3` and story posts in `facebook-5`, HxAgent performs reasonably well but with limited consistency. Tasks requiring the selection of hidden options (`flight-Alaska-auto`) or actions that span multiple UI elements (e.g., date-pickers in `expedia-3` and `expedia-5`) achieve only 30% `Exact-Match`.

Table 7. Success Rates of Generated Test Scripts (RQ2)

Test scripts for	Valid / Successful Trial
MiniWoB++	1048/1048 = 100.0%
Real-world Application	276/304 = 90.8%
OnlineMind2Web	109/141 = 77.3%

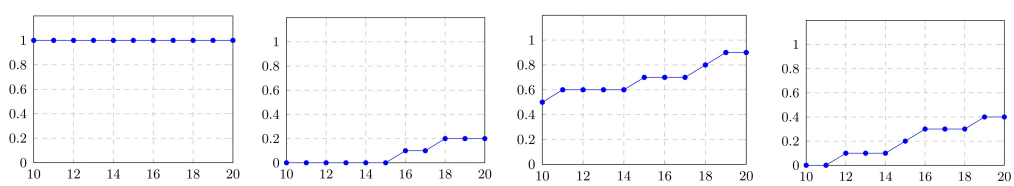
Similarly, for tasks involving complex forms (e.g., `stackoverflow-3`), the system generated invalid data during form filling.

7.2 Test Case Generation (RQ2)

We used HxAgent to generate test scripts for regression testing for the 1,493 tasks with correct sequences of actions in RQ1. We then executed them on the Web applications under test. Although test scripts are interpreted one-to-one from the SoA, they may fail during execution due to various reasons. Thus, a test script is valid if it (1) executes all steps without errors, and (2) successfully achieves the intended testing goal. As seen in Table 7, all successful SoAs in the MiniWoB++ dataset produced valid test scripts. For the real-world applications, 90.8% of successful SoA led to valid test scripts. Simulated MiniWoB++ environment comprises static, controlled HTML pages, making execution more predictable. In contrast, real-world applications often involve dynamic content and runtime variability, which can cause scripts to fail. For example, changing layouts or search result content, which can lead to inconsistent/invalid XPath locators. For LinkedIn, the same "Jobs" button may produce different XPath locators depending on the page's layout during action sequence generation, resulting in invalid tests. For Expedia, where fluctuating network latency caused the test script to attempt a click flight before the search results were fully loaded, leading to failure.

On the OnlineMind2Web dataset, a success rate of 77.3% is achieved. While this dataset has many tasks similar to those of the Real-world dataset, it also covers a diverse range of website categories, and failures often occur on sites with dynamic banners, dropdowns with dynamic content, or frequently changing navigation paths. For example, when searching for "SAM2" on GitHub, the generated script successfully reaches the listing page during execution. However, during evaluation, the same script may instead return a *429 Too Many Requests* HTTP request error, an instability caused by dynamic request limits. Solutions such as generating robust locators [34, 49] or selecting relevant ones [48] may help mitigate these issues.

7.3 Experience Analysis (RQ3)



(a) click-button (b) Alaska-auto (c) search-engine (d) search w/o rules

Fig. 8. Moving average of selected tasks in training (RQ3).

Fig. 8 displays the moving average for Experience, i.e., *the average of the correct SoA generated over a window of 10 recent runs during the training phase.* Comparing Figures 8c and 8d (without experience rules), the moving average in Figure 8c starts at 0.5, indicating 5 correct action sequences over 10 runs, while in Figure 8d, the starting point is 0, with no correct sequences. Consequently, after

Table 8. Contribution of Different Components (RQ4).

Module	Exact-Match (%)	Prefix-Match (%)
HxAgent (4o)	**97**	**99**
w/o experience	88	93
w/o (SAM + experience)	47	67
w/o iterative planning	75	85

20 runs, the agent *without experience rules* reaches only a 0.4 moving average (4 correct sequences over the last 10 runs) compared to a 0.9 moving average (*9 correct sequences over the last 10 runs*) as experience rules are included. As shown, the increase in the moving average over subsequent runs indicates that HxAgent *keeps acquiring more and more correct knowledge* during training. For `click-button` with the best possible initial performance (Fig. 8a), the knowledge retention remains consistent during training with 100% moving average. However, for a complex task (`Alaska-auto`), the moving average is lower as it is harder to obtain the correct sequence. Tracking the moving average during the training phase helps us monitor the quality of the knowledge in Experience and select the best set of few-shot examples and textual rules in prediction. We can halt the training after a certain number of runs if the moving average exceeds a threshold (Fig. 8c), and then capture the knowledge at that point in Experience prompts for next predictions.

7.4 Ablation Study (RQ4)

Ablation Study on Key Components. We removed HxAgent's key components and compared their performance. For each component, we removed the corresponding section in the main prompt. However, for the variant *w/o iterative planning*, we changed the main prompt to ask LLM only once to generate a complete SoA without the loop for planning. As seen in Table 8, all variants have lower accuracies than HxAgent, indicating the contributions of each component. Without sequence actions memory, the variant `w/o (SAM + experience)` has the most drop in `Exact-Match` (47 vs. 97). Thus, SAM is most important with its retaining the history of previous steps and evolving states to decide the next action. We found that `w/o (SAM + experience)` kept repeating some action multiple times until the step limit. HxAgent *w/o iterative agent planning* exhibits a decrease of 23% in `Exact-Match` where it struggles on tasks with multiple screens. The variant *w/o experience* has the smallest drop of 9% in `Exact-Match`. *This confirms our idea that experience with reflection (used in Li* et al.*) has less impact than the combination of* `SAM+experience` *and iterative planning.*

Ablation Study on Varying Sizes of Short-term Memory. Our results show that restricting memory to *only 1–3 most recent actions* reduces `Exact-Match` by 6% and 3%, respectively. Full access to short-term memory proves beneficial for tasks where the current state does not fully reflect the completion of prior steps. For example, in the `click-tab-2` task, full memory access enables HxAgent to track all visited tabs, preventing redundant clicks on the same tab—an issue with limited memory.

7.5 Cost Analysis (RQ5)

Figure 9 shows the token distributions in the prompts and generated outputs for each component of HxAgent: *content tokens* from the Content Extractor module, *agent tokens* from the Agent module, and *rule tokens* from the Rule Extractor module. These components are invoked multiple times during the iterative loop. The number of tokens in the prompts varies based on the complexity of the web application and the length of the action sequence. Among the components, the Agent module consumes the most tokens, followed by the Content Extractor module, as both manage the web state and play critical roles in the iterative loop. Figure 10 displays the token consumption for the input generator in HxAgent in the Agent component. The input generator is called a fixed

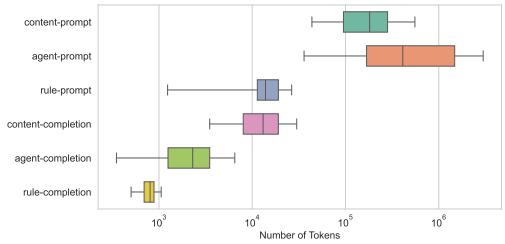


Fig. 9. #tokens for each component's input/output.

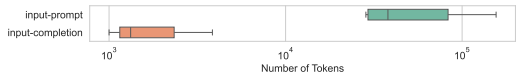


Fig. 10. #tokens for input generator (log scale).

number of times, depending on the number of input fields in the tasks. In summary, the average cost for all tasks is 80k input and 22k output tokens for both training and evaluation phases.

7.6 Threats to Validity

There are several concerns that might affect the validity of this study. One internal threat to validity is concerned with the hallucination of the LLMs as their outputs are not guaranteed to be consistent between different runs [26]. To alleviate this concern, we set the temperature parameter, which affects the randomness of generated responses of the LLMs, to zero to produce a more consistent and deterministic output. We also defined the prompts to be focused and to request specific outputs. As a result, our experiments showed consistency in the generated outputs by the LLMs.

The use of three datasets may limit the generalizability of our study. The applications in those datasets might not be representative for Web apps, and their UI elements and interactions may not reflect all types of elements found in general applications. Nonetheless, the MiniWoB++ dataset is widely used as a benchmark for automating computer tasks in previous studies (e.g., [17, 35, 38, 61, 71]). The Online-Mind2Web includes tasks from 136 real-world websites, covering diverse scenarios that users perform online. Seven real-world applications in our dataset are popular from various domains from social networks (Facebook, LinkedIn) to e-commerce (Amazon) to professional support (Stackoverflow). They are modern and well-updated applications that use common and diverse sets of UI elements. They include UI elements and actions that are typically found in modern web-based applications today. Taken together, the combination of MiniWoB++, Online-Mind2Web, and our curated dataset provides both controlled dynamic tasks and realistic real-world applications, offering a broad and meaningful representation of modern web environments.

8 Related Work

Benchmarks for Web Task Completion and UI Agents. The rapid progress of web UI agents has been driven by the availability of realistic benchmarks for navigation and interaction. WebArena [73] and WebLINX [46] offer controlled environments for natural language–based tasks, while AssistantBench [1] emphasizes long-horizon, multi-step workflows. Other large-scale efforts—including WebGames [62], OSUniverse [13], BEARCUBS [58], EconWebArena [45], and BrowseComp [3]—explore diverse challenges such as interactive game-like tasks, desktop-scale GUIs, live web browsing, economic decision-making, and multi-hop reasoning. Despite their breadth, these benchmarks typically report only task completion accuracy and do not provide ground-truth sequences of actions (SoAs), which limits their applicability for automated test script generation. Mind2Web [14] addresses this gap by collecting human demonstration trajectories, but these are expressed in natural language rather than machine-executable form. *Therefore, we did not use the above benchmarks. To bridge this limitation, we collected our own dataset with real-world websites*

(Table 1), and used MiniWoB++ [38] and OnlineMind2Web [66] as both are recent benchmarks and align with test generation with available or easily enhanced SoAs.

Web Task Completion Agents. Recent systems integrate diverse strategies: Beyond Browsing [2] mixes browser actions with API calls, ShowUI [5] grounds screenshots for zero-shot generalization, and OpenAI's Operator [4] demonstrates industrial-scale multimodal browser control. *Yet these agents only report task completion and do not explicitly produce SoAs, therefore, we did not compare them with HxAgent in test generation.*

Web UI Agents. Multimodal UI agents combine DOM structures with screenshots to improve grounding (e.g., SeeClick [10], CogAgent [25]). Recent advances include reflection-based reasoning (InfiGUIAgent [40]), iterative screenshot-only interaction (UI-TARS [52]), reinforcement learning strategies (UI-AGILE [36]), and trajectory alignment (UI-Venus [18]). However, most *represent the actions as absolute coordinates or textual descriptions*, which are brittle for regression testing and not directly executable. *We did not include these approaches in our comparison, as their outputs (textual descriptions or absolute coordinates) would require conversion into executable actions for test generation, a process that could introduce errors and lead to unfair comparisons.* We thus use SeeAct [70] as a baseline, as *it can prompt LLMs to output executable SoAs* and it was shown to outperform the state-of-the-art approaches.

Computer Use Agents. Beyond the web, agents have been developed for mobile (Guardian [53], GPTDroid [42], AutoDroid [63], DroidAgent [68]) and desktop use (TaskMatrix.AI [37], Computer Use [22], Claude 3.5 Sonnet [6]). These systems extend LLM control to apps, files, and OS-level interactions, but like web task completion agents, *they focus on accomplishing tasks rather than producing machine-executable SoAs for automated testing* as in HxAgent.

Cucumber [11] with Gherkin was an early approach using texts for test flow creation, requiring manual step definitions. Later methods automatically generated test paths from navigation models [7, 56, 72] or human execution traces [16, 50]. KeyjaxTest [51] introduced keyword-guided exploration for targeted path generation. LLMs have enabled applications across mobile testing [41, 43, 44, 53, 64], unit testing [12, 33, 54, 69], test oracles [47], and web agents [8, 9, 23, 24, 32, 61].

9 Conclusion

Novelty. We present HxAgent, a multi-modal, iterative Web testing framework using LLMs with vision capabilities to generate Web test scripts for GUI tasks described in natural language. We found that our **iterative process** grounding the model's reasoning in the actual text and visual context at each step helps *reduce hallucinations in UI agents*. Moreover, we found that proactive correction in HxAgent continuously reassesses the state after each step to adjust its plan as needed. In contrast, the state-of-the-art self-reflection approaches use a reactive repair strategy, where corrections are attempted only after an entire action trajectory has failed. Our evaluation on three state-of-the-art datasets shows its superior performance over the baselines.

Impacts on Future Research. Looking ahead, this work opens promising directions for the future of automated specification and test generation. As LLMs and multi-modal reasoning continue to advance, we envision frameworks that not only generate reliable action sequences but also acquire deeper semantic understanding of software behavior, enabling proactive detection of subtle faults on Web application interfaces. Future research may move toward fully autonomous, self-reflective agents that continuously learn to test and verify diverse applications without human interventions. Moreover, the creation of open, large-scale, and dynamically evolving benchmarks will be essential to evaluate these systems under real-world conditions. Ultimately, these efforts point toward a future in which AI-driven testing agents can act as trustworthy collaborators, reducing human effort while improving software reliability.

10. Data Availability. Data/code are available on our website [28].

References

[1] Anonymous. 2024. AssistantBench: Can Web Agents Solve Realistic and Time-Consuming Tasks? *arXiv preprint* (2024).

[2] Anonymous. 2025. Beyond Browsing: API-Based Web Agents. *arXiv preprint arXiv:2502.xxxxx* (2025).

[3] Anonymous. 2025. BrowseComp: Agentic AI Web Benchmark. Technical Report / OpenAI.

[4] Anonymous. 2025. Operator: A Vision-Enhanced GPT Agent for Autonomous Web Interaction. *Technical Report* (2025).

[5] Anonymous. 2025. ShowUI: Compact Visual Grounding for Web Agents. *arXiv preprint arXiv:2503.xxxxx* (2025).

[6] Anthropic. 2025. Computer Use and AI Agents: A New Paradigm for Screen Interaction. *Technical Blog* (2025).

[7] Matteo Biagiola, Andrea Stocco, Filippo Ricca, and Paolo Tonella. 2019. Diversity-based web test generation. In *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 142–153.

[8] Qi Chen, Dileepa Pitawela, Chongyang Zhao, Gengze Zhou, Hsiang-Ting Chen, and Qi Wu. 2024. Webvln: Vision-and-language navigation on websites. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 1165–1173.

[9] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. [n. d.]. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.

[10] Kanzhi Cheng, Qiushi Sun, Yougang Chu, Fangzhi Xu, Li YanTao, Jianbing Zhang, and Zhiyong Wu. 2024. SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 9313–9332. doi:10.18653/v1/2024.acl-long.505

[11] Cucumber [n. d.]. Cucumber. https://cucumber.io/docs/gherkin/.

[12] Arghavan Moradi Dakhel, Amin Nikanjam, Vahid Majdinasab, Foutse Khomh, and Michel C Desmarais. 2024. Effective test generation using pre-trained large language models and mutation testing. *Information and Software Technology* 171 (2024), 107468.

[13] Mariya Davydova, Daniel Jeffries, Patrick Barker, Arturo Márquez Flores, and Sinéad Ryan. 2025. OSUniverse: Benchmark for Multimodal GUI-navigation AI Agents. *arXiv preprint* (2025).

[14] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Sam Stevens, Boshi Wang, Huan Sun, and Yu Su. 2024. Mind2web: Towards a generalist agent for the web. *Advances in Neural Information Processing Systems* 36 (2024).

[15] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, et al. [n. d.]. WorkArena: How Capable are Web Agents at Solving Common Knowledge Work Tasks?. In *ICLR 2024 Workshop on Large Language Model (LLM) Agents*.

[16] Markus Ermuth and Michael Pradel. 2016. Monkey see, monkey do: Effective generation of GUI tests with inferred macro events. In *Proceedings of the 25th International Symposium on Software Testing and Analysis*. 82–93.

[17] Hiroki Furuta, Kuang-Huei Lee, Ofir Nachum, Yutaka Matsuo, Aleksandra Faust, Shixiang Shane Gu, and Izzeddin Gur. 2023. Multimodal Web Navigation with Instruction-Finetuned Foundation Models. In *The Twelfth International Conference on Learning Representations*.

[18] Zhangxuan Gu et al. 2025. UI-Venus Technical Report: Building High-performance UI Agents with RFT. *arXiv preprint arXiv:2508.10833* (2025).

[19] Izzeddin Gur, Hiroki Furuta, Austin V Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2023. A Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis. In *The Twelfth International Conference on Learning Representations*.

[20] Izzeddin Gur, Hiroki Furuta, Austin V Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. 2024. A Real-World WebAgent with Planning, Long Context Understanding, and Program Synthesis. In *The Twelfth International Conference on Learning Representations*. https://openreview.net/forum?id=9JQtrumvg8

[21] Izzeddin Gur, Ofir Nachum, Yingjie Miao, Mustafa Safdari, Austin Huang, Aakanksha Chowdhery, Sharan Narang, Noah Fiedel, and Aleksandra Faust. 2023. Understanding HTML with Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali (Eds.). Association for Computational Linguistics, Singapore, 2803–2821. doi:10.18653/v1/2023.findings-emnlp.185

[22] Izzeddin Gur, Xinyun Zhang, Xinyang Chen, Shuyan Zhou, Adams Wei Yu, Kevin Lin, and Karthik Narasimhan. 2023. Computer Use Agents: Benchmarking autonomous computer use by language models. *arXiv preprint arXiv:2307.09288* (2023).

[23] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. 2024. WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Lun-Wei Ku, Andre Martins, and Vivek Srikumar (Eds.). Association for Computational Linguistics, Bangkok, Thailand, 6864–6890. doi:10.18653/v1/2024.acl-long.371

[24] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxiao Dong, Ming Ding, et al. 2024. Cogagent: A visual language model for gui agents. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14281–14290.

[25] Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. 2024. CogAgent: A Visual Language Model for GUI Agents. arXiv:2312.08914 [cs.CV] https://arxiv.org/abs/2312.08914

[26] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. *ACM Transactions on Information Systems* 43, 2 (Jan. 2025), 1–55. doi:10.1145/3703155

[27] Peter C Humphreys, David Raposo, Tobias Pohlen, Gregory Thornton, Rachita Chhaparia, Alistair Muldal, Josh Abramson, Petko Georgiev, Adam Santoro, and Timothy Lillicrap. 2022. A data-driven approach for learning to control computers. In *International Conference on Machine Learning*. PMLR, 9466–9482.

[28] HxAgent [n. d.]. HxAgent. https://anonymous.4open.science/r/hxagent.

[29] Sheng Jia, Jamie Ryan Kiros, and Jimmy Ba. 2018. DOM-Q-NET: Grounded RL on Structured Language. In *International Conference on Learning Representations*.

[30] Katalon [n. d.]. Katalon. https://katalon.com/.

[31] Geunwoo Kim, Pierre Baldi, and Stephen McAleer. 2024. Language models can solve computer tasks. *Advances in Neural Information Processing Systems* 36 (2024).

[32] Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, et al. 2024. AutoWebGLM: A Large Language Model-based Web Navigating Agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 5295–5306.

[33] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K Lahiri, and Siddhartha Sen. 2023. Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 919–931.

[34] Maurizio Leotta, Andrea Stocco, Filippo Ricca, and Paolo Tonella. 2016. ROBULA+: An algorithm for generating robust XPath locators for web testing. *Journal of Software: Evolution and Process* 28, 3 (2016), 177–204.

[35] Tao Li, Gang Li, Zhiwei Deng, Bryan Wang, and Yang Li. 2023. A Zero-Shot Language Agent for Computer Control with Structured Reflection. In *The 2023 Conference on Empirical Methods in Natural Language Processing*.

[36] Shuquan Lian et al. 2025. UI-AGILE: Advancing GUI Agents with Effective Reinforcement Learning and Precise Inference-Time Grounding. *arXiv preprint arXiv:2507.22025* (2025).

[37] Yuxi Liang, Zhuo Chen, Shuyan Zhou, Adams Wei Yu, Chenfei Wu, Xing Xie, and Jiang Bian. 2024. TaskMatrix.AI: Towards building generalist agents for billions of tasks. *arXiv preprint arXiv:2303.16434* (2024).

[38] Evan Zheran Liu, Kelvin Guu, Panupong Pasupat, Tianlin Shi, and Percy Liang. 2018. Reinforcement Learning on Web Interfaces using Workflow-Guided Exploration. In *International Conference on Learning Representations (ICLR)*.

[39] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. 2023. AgentBench: Evaluating LLMs as Agents. In *The Twelfth International Conference on Learning Representations*.

[40] Yuhang Liu et al. 2025. InfiGUIAgent: A Multimodal Generalist GUI Agent with Native Reasoning and Reflection. *arXiv preprint arXiv:2501.04575* (2025).

[41] Zhe Liu, Chunyang Chen, Junjie Wang, Xing Che, Yuekai Huang, Jun Hu, and Qing Wang. 2023. Fill in the blank: Context-aware automated text input generation for mobile gui testing. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 1355–1367.

[42] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2023. Chatting with GPT-3 for Zero-Shot Human-Like Mobile Automated GUI Testing. arXiv:2305.09434 [cs.SE] https://arxiv.org/abs/2305.09434

[43] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Xing Che, Dandan Wang, and Qing Wang. 2024. Make llm a testing expert: Bringing human-like interaction to mobile gui testing via functionality-aware decisions. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–13.

[44] Zhe Liu, Chunyang Chen, Junjie Wang, Mengzhuo Chen, Boyu Wu, Zhilin Tian, Yuekai Huang, Jun Hu, and Qing Wang. 2024. Testing the limits: Unusual text inputs generation for mobile app crash detection with large language model. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*. 1–12.

[45] Zefang Liu and Yinzhu Quan. 2025. EconWebArena: Benchmarking Autonomous Agents on Economic Tasks in Realistic Web Environments. *arXiv preprint* (2025).

[46] Xing Han Lù, Zdeněk Kasner, and Siva Reddy. 2024. WebLINX: Real-World Website Navigation with Multi-Turn Dialogue. arXiv:2402.05930 [cs.CL]

[47] Noor Nashid, Mifta Sintaha, and Ali Mesbah. 2023. Retrieval-based prompt selection for code-related few-shot learning. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2450–2462.

[48] Michel Nass, Emil Alégroth, Robert Feldt, Maurizio Leotta, and Filippo Ricca. 2023. Similarity-based web element localization for robust test automation. *ACM Transactions on Software Engineering and Methodology* 32, 3 (2023), 1–30.

[49] Vu Nguyen, Thanh To, and Gia-Han Diep. 2021. Generating and selecting resilient and maintainable locators for Web automated testing. *Software Testing, Verification and Reliability* 31, 3 (2021), e1760.

[50] Ana CR Paiva, André Restivo, and Sérgio Almeida. 2020. Test case generation based on mutations over user execution traces. *Software quality journal* 28, 3 (2020), 1173–1186.

[51] Xiao-Fang Qi, Yun-Long Hua, Peng Wang, and Zi-Yuan Wang. 2019. Leveraging keyword-guided exploration to build test models for web applications. *Information and Software Technology* 111 (2019), 110–119.

[52] Yujia Qin, Yining Ye, Junjie Fang, Haoming Wang, Shihao Liang, Shizuo Tian, Junda Zhang, Jiahao Li, Yunxin Li, Shijue Huang, Wanjun Zhong, Kuanye Li, Jiale Yang, Yu Miao, Woyu Lin, Longxiang Liu, Xu Jiang, Qianli Ma, Jingyu Li, Xiaojun Xiao, Kai Cai, Chuang Li, Yaowei Zheng, Chaolin Jin, Chen Li, Xiao Zhou, Minchao Wang, Haoli Chen, Zhaojian Li, Haihua Yang, Haifeng Liu, Feng Lin, Tao Peng, Xin Liu, and Guang Shi. 2025. UI-TARS: Pioneering Automated GUI Interaction with Native Agents. arXiv:2501.12326 [cs.AI] https://arxiv.org/abs/2501.12326

[53] Dezhi Ran, Hao Wang, Zihe Song, Mengzhou Wu, Yuan Cao, Ying Zhang, Wei Yang, and Tao Xie. 2024. Guardian: A Runtime Framework for LLM-Based UI Exploration. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 958–970.

[54] Nikitha Rao, Kush Jain, Uri Alon, Claire Le Goues, and Vincent J Hellendoorn. 2023. CAT-LM training language models on aligned code and tests. In *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 409–420.

[55] Selenium [n. d.]. Selenium. https://www.selenium.dev/.

[56] Salman Sherin, Asmar Muqeet, Muhammad Uzair Khan, and Muhammad Zohaib Iqbal. 2023. QExplore: An exploration strategy for dynamic web applications using guided search. *Journal of Systems and Software* 195 (2023), 111512.

[57] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems* 36 (2024).

[58] Yixiao Song, Katherine Thai, Chau Minh Pham, Yapei Chang, Mazin Nadaf, and Mohit Iyyer. 2025. BEARCUBS: A Benchmark for Computer-Using Web Agents. *arXiv preprint* (2025).

[59] Theodore Sumers, Shunyu Yao, Karthik Narasimhan, and Thomas Griffiths. [n. d.]. Cognitive Architectures for Language Agents. *Transactions on Machine Learning Research* ([n. d.]).

[60] Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. 2024. Adaplanner: Adaptive planning from feedback with language models. *Advances in Neural Information Processing Systems* 36 (2024).

[61] Lucas-Andrei Thil, Mirela Popa, and Gerasimos Spanakis. 2024. Navigating WebAI: Training Agents to Complete Web Tasks with Large Language Models and Reinforcement Learning. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*. 866–874.

[62] George Thomas, Alex J. Chan, Jikun Kang, Wenqi Wu, Filippos Christianos, Fraser Greenlee, Andy Toulis, and Marvin Purtorab. 2025. WebGames: Challenging General-Purpose Web-Browsing AI Agents. *arXiv preprint* (2025).

[63] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2023. AutoDroid: LLM-powered Task Automation in Android. *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking* (2023). https://api.semanticscholar.org/CorpusID:261277501

[64] Hao Wen, Yuanchun Li, Guohong Liu, Shanhui Zhao, Tao Yu, Toby Jia-Jun Li, Shiqi Jiang, Yunhao Liu, Yaqin Zhang, and Yunxin Liu. 2024. Autodroid: Llm-powered task automation in android. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 543–557.

[65] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An Illusion of Progress? Assessing the Current State of Web Agents. (2025). arXiv:2504.01382 [cs.AI] https://arxiv.org/abs/2504.01382

[66] Tianci Xue, Weijian Qi, Tianneng Shi, Chan Hee Song, Boyu Gou, Dawn Song, Huan Sun, and Yu Su. 2025. An Illusion of Progress? Assessing the Current State of Web Agents. arXiv:2504.01382 [cs.AI] https://arxiv.org/abs/2504.01382

[67] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing Reasoning and Acting in Language Models. In *International Conference on Learning Representations (ICLR)*.

[68] Juyeon Yoon, Robert Feldt, and Shin Yoo. 2023. Autonomous Large Language Model Agents Enabling Intent-Driven Mobile GUI Testing. arXiv:2311.08649 [cs.SE] https://arxiv.org/abs/2311.08649

[69] Zhiqiang Yuan, Mingwei Liu, Shiji Ding, Kaixin Wang, Yixuan Chen, Xin Peng, and Yiling Lou. 2024. Evaluating and Improving ChatGPT for Unit Test Generation. *Proceedings of the ACM on Software Engineering* 1, FSE (2024), 1703–1726.

[70] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. 2024. GPT-4V(ision) is a Generalist Web Agent, if Grounded. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*, Ruslan Salakhutdinov, Zico Kolter, Katherine Heller, Adrian Weller, Nuria Oliver, Jonathan Scarlett,

and Felix Berkenkamp (Eds.). PMLR, 61349–61385. https://proceedings.mlr.press/v235/zheng24e.html

[71] Longtao Zheng, Rundong Wang, Xinrun Wang, and Bo An. 2023. Synapse: Trajectory-as-exemplar prompting with memory for computer control. In *The Twelfth International Conference on Learning Representations*.

[72] Yan Zheng, Yi Liu, Xiaofei Xie, Yepang Liu, Lei Ma, Jianye Hao, and Yang Liu. 2021. Automatic web testing using curiosity-driven reinforcement learning. In *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 423–435.

[73] Shuyan Zhou, Frank F Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, et al. 2023. WebArena: A Realistic Web Environment for Building Autonomous Agents. *arXiv preprint arXiv:2307.13854* (2023). https://webarena.dev